

Controle de Refeitório

Modelo de Dados

SQL Server — banco GIPP, schema dbo
Cinco tabelas, uma view, detalhamento coluna por coluna

BANCO	GIPP (SQL Server), schema dbo
OBJETOS	meal_log · meal_diner_group · meal_biometric · meal_enroll_token · meal_coupon · view vw_meal_diner
MÓDULO DA API	src/modules/meal — rotas /api/v1/gipp/meal/*
FONTES EXTERNAS	TMPPRD12 (Protheus), CONSINCO, MySQL global (usuários e permissões)
METADADOS LIDOS EM	28/08/2026, direto de sys.columns, sys.check_constraints, sys.indexes e sys.foreign_keys da instância
VOLUME NA LEITURA	meal_log 27 · meal_biometric 5 · meal_enroll_token 6 · meal_diner_group 3 · meal_coupon 0 (meal_log é tabela viva: recebeu uma linha durante a redação deste documento)

Como este documento foi produzido. A estrutura veio das visões de catálogo do próprio SQL Server; o significado de cada coluna veio do código que a escreve (`src/modules/meal`) e dos comentários do script de deploy; o comportamento de fuso horário da seção 3.4 foi medido contra a instância viva, não deduzido. Onde código, banco e documentação existente divergem, a divergência está listada na seção 14.

Sumário

1	Visão geral — o que cada tabela resolve
2	Mapa de relacionamentos
3	Convenções transversais — chave do comensal, snapshot, fusos, idempotência
4	Domínios codificados — o significado de cada número
5	meal_log — a refeição servida
6	meal_diner_group — os grupos sem matrícula
7	meal_biometric — o vetor facial e o consentimento
8	meal_enroll_token — o convite de autocadastro
9	meal_coupon — o saldo do cupom fiscal
10	vw_meal_diner — quem pode ser servido
11	Objetos externos referenciados sem chave estrangeira
12	Fluxos — que linhas nascem em cada operação
13	Ciclo de vida, retenção e LGPD
14	Observações técnicas e divergências encontradas
A	Anexo — DDL reconstruído a partir do catálogo
B	Anexo — consultas de verificação

1. Visão geral — o que cada tabela resolve

O módulo tem uma responsabilidade só: registrar quem foi servido, para contar refeições, ratear custo por centro de custo e faturar contra terceiro quando a refeição não é do grupo. Quem é a pessoa, qual o centro de custo e qual a filial são lidos do Protheus em tempo real — **nenhuma tabela deste módulo copia cadastro de pessoal**.

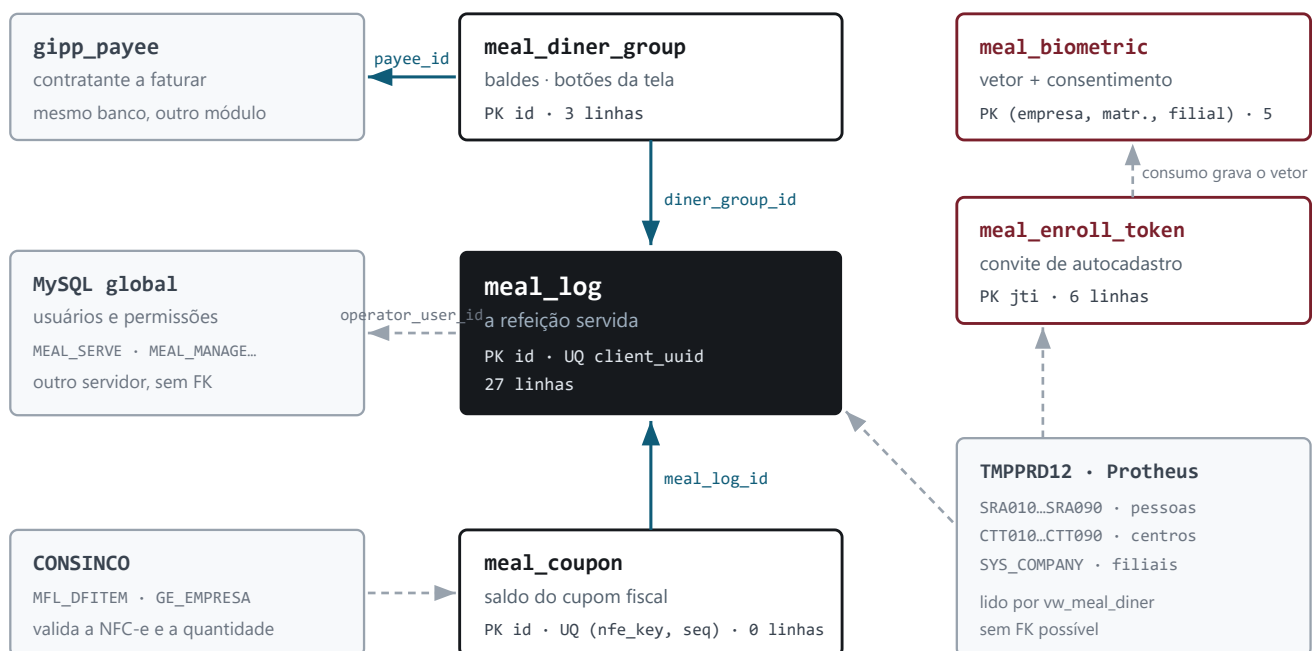
TABELA	NATUREZA	RESPONSABILIDADE ÚNICA	LINHAS
meal_log	Fato, append-only	Uma linha por refeição servida. É a única tabela que os relatórios agregam, e é documento de custo e de auditoria.	27
meal_diner_group	Cadastro	Os "baldes": grupos contados sem matrícula (Jovem Aprendiz, Segurança, Visitante). Viram os botões da tela do operador.	3
meal_biometric	Dado pessoal sensível	Um vetor facial por pessoa, mais a prova do consentimento. Sem coluna de imagem, por desenho.	5
meal_enroll_token	Estado transitório	O convite individual de autocadastro facial: emissão, tentativas, consumo e queima.	6
meal_coupon	Saldo	Uma linha por refeição consumida de um cupom fiscal (NFC-e) de prestador de serviço. É controle de saldo, não blacklist de chave.	0

O que não existe, e é de propósito

- **Tabela de colaboradores.** A fonte é o Protheus, via `vw_meal_diner`. Cópia envelhece; view não.
- **Tabela de lojas ou de “filiais habilitadas”.** O cadastro é lido de `TMPPRD12.dbo.SYS_COMPANY` em tempo real. Uma versão anterior do script chegou a criar `meal_pos_site`; o deploy de 27/08/2026 a descarta.
- **Coluna de imagem em `meal_biometric`.** A promessa “só o vetor” é restrição de schema, não disciplina de programador.
- **Coluna `is_duplicate` em `meal_log`.** Duplicata é calculada no relatório, para que uma revisão da regra valha também para o histórico.
- **Chave estrangeira para o Protheus.** FK entre bancos diferentes não existe no SQL Server; sobra validação de formato por `CHECK` e conferência na abertura da sessão do operador.

2. Mapa de relacionamentos

Existem três chaves estrangeiras, todas com `ON DELETE NO ACTION`: refeição registrada não pode desaparecer levando o histórico junto. As linhas tracejadas são referências **lógicas** — colunas que apontam para dados em outro banco, onde FK não é possível.



linha cheia = chave estrangeira real · linha tracejada = referência lógica, sem FK

3. Convenções transversais

Cinco decisões atravessam todas as tabelas. Entendê-las antes das colunas evita ler cada coluna duas vezes.

3.1 A chave do comensal tem três partes

Não existe “id do colaborador”. A identidade de uma pessoa é sempre a trinca do Protheus, e meia chave não serve:

COLUNA	ORIGEM NO PROTHEUS	REGRA
<code>company_code</code>	sufixo da tabela <code>SRAxxx</code>	Dois dígitos: <code>01</code> , <code>02</code> , <code>03</code> , <code>06</code> , <code>07</code> , <code>08</code> , <code>09</code> . A matrícula só é única <i>dentro</i> de uma empresa — sem esta coluna, duas pessoas diferentes colidem.
<code>employee_id</code>	<code>RA_MAT</code>	Até 6 posições, com os zeros à esquerda preservados (<code>001981</code> , não <code>1981</code>). A view aplica <code>LTRIM(RTRIM())</code> na projeção.
<code>branch_code</code>	<code>RA_FILIAL</code>	Quatro dígitos. É a lotação da pessoa, não onde ela comeu.

A mesma trinca é a PK de `meal_biometric`, repete-se em `meal_enroll_token`, e aparece duas vezes em `meal_log` — uma para quem comeu, outra (`host_*`) para quem convidou.

3.2 `site_code` e `branch_code` não são a mesma coisa

`branch_code` é **onde a pessoa é lotada**. `site_code` é **onde a refeição foi servida**. Os dois são o `M0_CODFIL` de quatro dígitos, e por isso é fácil confundir-los — mas divergem em caso real: existem duas filiais chamadas Interlagos, e gente lotada em `0203` almoça no `0202`. A linha `id = 99` de `meal_log` é exatamente isso: `branch_code = 0209`, `site_code = 0202`.

Consequência prática: relatório por loja agrupa por `site_code`; rateio de custo vai por `company_code + cost_center`. Trocar um pelo outro produz número plausível e errado.

3.3 Snapshot ou lookup — a regra de decisão

Algumas colunas guardam o valor no momento do fato; outras são buscadas na hora do relatório. O critério é único: **valor contábil é congelado, rótulo é buscado**.

CONGELADO NO MOMENTO DO FATO	BUSCADO NA HORA DA LEITURA
<code>meal_log.cost_center</code> — o <code>RA_CC</code> vigente quando a refeição foi gravada. É o que o rateio soma.	<code>cost_center_description</code> — descrição vem por <code>OUTER APPLY</code> na view. Descrição é rótulo.
<code>meal_log.service_date</code> — data civil gravada, não derivada de <code>served_at</code> na consulta.	<code>employee_name</code> — nunca copiado para <code>meal_log</code> . Sai da view a cada leitura.
<code>meal_coupon.meals_authorized</code> — quantidade lida do Consinco no primeiro resgate; não recalculada depois.	Situação de desligamento — <code>is_terminated</code> / <code>terminated_at</code> vêm da view, sempre atuais.
<code>meal_coupon.seqproduto</code> e <code>tp_emis</code> — o código do produto muda com o tempo; o relatório de	<code>payee_name</code> / <code>payee_document</code> — <code>LEFT JOIN</code> em <code>gipp_payee</code> no relatório de faturamento.

seis meses atrás precisa continuar explicável.

Limitação conhecida e assumida: se a pessoa for transferida entre servir e sincronizar a fila offline, o `cost_center` gravado é o do momento do sincronismo. Na prática a fila descarrega no mesmo dia, e a alternativa — confiar no centro de custo enviado pelo aparelho — permitiria rateio inventado.

3.4 Fuso horário: quem escreve UTC e quem escreve hora local

Este é o ponto mais escorregadio do modelo, e foi medido, não deduzido

As colunas `datetime2(0)` deste módulo **não estão todas no mesmo fuso**. O driver `mssql` converte um `Date` do Node para UTC ao gravar (`useUTC` vem ligado por padrão), enquanto `SYSDATETIME()` devolve a hora **local** do SQL Server (UTC-3). Medição feita contra a instância em 28/08/2026:

```
node local    → 2026-08-28 10:16:18      -- hora de Brasília
node ISO      → 2026-08-28T13:16:18Z
gravado       → 2026-08-28 13:16:18      -- o driver gravou o UTC
SYSDATETIME() → 2026-08-28 10:28:20      -- o SQL Server devolve local
```

Portanto: **toda coluna escrita pela aplicação está em UTC; toda coluna escrita por `DEFAULT SYSDATETIME()` ou por `SET ... = SYSDATETIME()` está em UTC-3**. Ler as duas no mesmo relatório sem converter produz diferença de três horas.

COLUNA	QUEM ESCRIVE	FUSO	OBSERVAÇÃO
<code>meal_log.served_at</code>	Aplicação	UTC	Relógio do aparelho, com correção só para o futuro.
<code>meal_log.service_date</code>	Aplicação	local	Data civil montada com <code>getFullYear/Month/Date</code> — partes locais do mesmo instante.
<code>meal_log.synced_at</code>	Aplicação	UTC	Só preenchida quando a linha veio pela fila.
<code>meal_biometric.enrolled_at</code>	Aplicação (parâmetro)	UTC	O <code>DEFAULT SYSDATETIME()</code> existe mas não é usado pelo <code>MERGE</code> — ver 3.4.1.
<code>meal_biometric.consent_at</code>	Aplicação	UTC	Sai do mesmo <code>new Date()</code> de <code>enrolled_at</code> .
<code>meal_biometric.revoked_at</code>	Aplicação (parâmetro)	UTC	Rebaixada a <code>enrolled_at</code> quando vier antes dele.
<code>meal_enroll_token.issued_at</code>	<code>DEFAULT SYSDATETIME()</code>	local	—
<code>meal_enroll_token.expires_at</code>	Aplicação	UTC	Mistura de fusos no mesmo par de colunas — ver observação 14.1.
<code>meal_enroll_token.consumed_at</code> , <code>burned_at</code>	<code>SYSDATETIME()</code> no <code>UPDATE</code>	local	—
<code>meal_coupon.redeemed_at</code>	Aplicação (parâmetro)	UTC	O <code>DEFAULT SYSDATETIME()</code> permanece na coluna e vale para <code>INSERT</code> feito fora da aplicação.
<code>meal_diner_group.created_at</code> , <code>updated_at</code>	<code>SYSDATETIME()</code>	local	—

3.4.1 POR QUE `ENROLLED_AT` VEM POR PARÂMETRO

Em 19/08/2026 todo cadastro facial passou a ser reprovado por `CK_meal_biometric_consent_before_enroll`. A causa: `consent_at` vinha do relógio do Node e `enrolled_at` do `DEFAULT SYSDATETIME()` — a constraint comparava **dois relógios**, e poucos segundos de dessincronia entre as máquinas reprovavam a operação, com a mensagem apontando para a constraint em vez de para a causa. Hoje os dois valores saem do mesmo `new Date()`, e a constraint volta a medir o que deveria: a ordem dos eventos. A mesma armadilha, espelhada, motivou o `CASE` em `revoked_at`.

3.5 Idempotência e concorrência estão no banco, não no código

MECANISMO	O QUE ELE IMPEDE, E POR QUE UM <code>SELECT</code> ANTES NÃO RESOLVERIA
<code>UX_meal_log_client</code> único em <code>client_uuid</code>	A fila offline reenvia a mesma refeição. O <code>INSERT ... WHERE NOT EXISTS</code> cobre o caso normal; em corrida entre dois envios simultâneos o índice único recusa o segundo (erro 2601/2627) e o repositório reconsulta e devolve a linha existente. O cliente não distingue os dois casos.
<code>UX_meal_coupon_seq</code> único em <code>(nfe_key , seq)</code>	Dois terminais lendo o mesmo QR no mesmo instante calculam o mesmo <code>MAX(seq)+1</code> . Um <code>SELECT</code> antes do <code>INSERT</code> não resolve: os dois leriam o mesmo saldo e os dois passariam. O <code>WITH (UPDLCK, HOLDLCK)</code> na leitura evita que a violação seja acionada o tempo todo; o índice único é a garantia final.
<code>CK_meal_coupon_seq</code> <code>seq <= meals_authorized</code>	Um defeito de aplicação que peça a quinta refeição de um cupom de quatro esbarra no banco. É a diferença entre “a regra está no código” e “a regra está garantida”.
<code>UX_meal_enroll_token_open</code> único filtrado, <code>consumed_at IS NULL</code> <code>AND burned_at IS NULL</code>	Duas pessoas não podem ter dois convites vivos para a mesma matrícula — dois links válidos significam dois caminhos para o mesmo cadastro e nenhum jeito de saber qual foi usado. Reemitir exige queimar o anterior.
<code>UPDATE ... OUTPUT</code> em <code>attempts</code>	Incremento e leitura na mesma instrução. Em duas requisições separadas, duas tentativas simultâneas leriam o mesmo valor e gastariam uma só — com 3 tentativas no total, essa corrida é a diferença entre 3 e infinitas.

4. Domínios codificados — o significado de cada número

Os valores abaixo estão espelhados em dois lugares: nas constraints do banco e em `src/modules/meal/domain/meal.enums.js`. Trocar um número num lugar só produz `INSERT` recusado pelo banco — o comportamento desejado, mas o lugar certo de mudar é nos dois.

diner_type — quem comeu

VALOR	SIGNIFICADO	COLUNAS OBRIGATÓRIAS E PROIBIDAS (<code>CK_MEAL_LOG_DINER</code>)
1	Colaborador	Exige <code>company_code</code> , <code>employee_id</code> , <code>branch_code</code> . Proíbe <code>diner_group_id</code> .
2	Grupo sem matrícula o “balde”	Exige <code>diner_group_id</code> . Proíbe a trinca do colaborador. É contado, não identificado.
3	Cupom fiscal prestador de serviço	Proíbe as quatro: sem matrícula e sem grupo. Ele pagou — a refeição dele é receita, não despesa, e por isso não cabe em “grupo”, que existe para contar quem come por conta da casa. Misturar os dois estragaria o rateio.

O ramo `diner_type = 3` foi acrescentado ao `CHECK` pelo deploy de 27/08/2026, em transação, com `WITH CHECK` — o predicado só foi *alargado*, então tudo que passava continua passando, e a constraint permanece confiável para o otimizador.

meal_type — qual refeição

1 almoço · 2 jantar · 3 café da manhã. Garantido por `CK_meal_log_meal_type`.

identified_by — como a pessoa foi identificada

Esta coluna **não é estatística**: é a prova, exigida pelo consentimento de LGPD, de que o caminho alternativo ao rosto existe e é usado. Se o relatório mostra refeições servidas por QR todos os dias, a liberdade de escolha está demonstrada por dado, não por declaração.

VALOR	CAMINHO	QUANDO É VÁLIDO
1	QR Code	Colaborador. É o caminho padrão.
2	Manual	Colaborador. Matrícula digitada pelo operador.
3	Facial	Colaborador. Único caso em que <code>match_score</code> pode vir preenchido.
4	Botão	Só grupo sem matrícula. Botão nunca registra uma matrícula.
5	Cupom fiscal	Só <code>diner_type = 3</code> .

`CK_meal_log_identified_by` garante apenas o *domínio* (1 a 5). O acoplamento entre `diner_type` e `identified_by` — balde só por botão, cupom só por cupom — é regra de aplicação em `meal-log.rules.js`: a constraint nunca cuidou disso, e alargar o escopo dela agora recusaria linhas já gravadas.

enroll_verified_by — como a identidade foi provada no cadastro facial

VALOR	CONSTANTE NO CÓDIGO	PROVA DE IDENTIDADE
-------	---------------------	---------------------

1	SIGNED_LINK	Link individual assinado: a pessoa digitou a própria data de nascimento, sozinha, no navegador.
2	IN_PERSON	Cadastro presencial no aparelho do operador — crachá apresentado, operador identificado olhando. Prova mais forte que a do link; é por isso que o valor é diferente. As 5 linhas existentes hoje são todas 2.
3	FIRST_MEAL_QR	Definido no domínio e aceito pela constraint, mas sem caminho que o grave no código atual — ver observação 14.6.

burned_reason — por que um convite foi queimado

Coluna livre `varchar(40)`; a aplicação grava exatamente três valores: `reissued` (o RH reemitiu, queimando o anterior), `birth_date_missing` (o Protheus não tem `RA_NASC` para conferir) e `attempts_exhausted` (as 3 tentativas acabaram). Nas 6 linhas de hoje só aparece `reissued`.

tp_emis — regime de emissão da NFC-e

1 normal · 9 contingência (`CK_meal_coupon_tp_emis`). Cupom em contingência é aceito desde que o documento esteja no Consinco, mas fica marcado: a venda ainda pode ser rejeitada pela SEFAZ, e o relatório de conferência precisa achar essas linhas.

5. meal_log — a refeição servida

Uma linha por refeição. É a tabela central: todos os relatórios agregam sobre ela, e é o documento de custo e de auditoria do módulo. Trata-se de tabela **append-only** — não há coluna de alteração nem de exclusão lógica, e nenhuma rota faz `UPDATE` nela.

COLUNA	TIPO	NULO	ESCRITO POR	SIGNIFICADO E REGRA
id PK	bigint	não	IDENTITY(1,1)	Chave técnica. Não é exposta ao aparelho — a identidade da refeição para o cliente é o <code>client_uuid</code> . Serve de alvo para <code>meal_coupon.meal_log_id</code> .
diner_type	tinyint	não	aplicação	1 colaborador · 2 grupo · 3 cupom. Define quais colunas de identificação são obrigatórias e quais são proibidas (seção 4).
company_code	varchar(10)	sim	aplicação	Empresa do Protheus, 2 dígitos. Obrigatória para <code>diner_type = 1</code> , proibida nos outros. A coluna aceita 10 caracteres, mas a aplicação exige o padrão <code>^[0-9]{2}\$</code> .
employee_id	varchar(6)	sim	aplicação	<code>RA_MAT</code> , com zeros à esquerda. O tamanho 6 é o limite físico da matrícula no Protheus.
branch_code	varchar(10)	sim	aplicação	<code>RA_FILIAL</code> — a lotação da pessoa, 4 dígitos. Não confundir com <code>site_code</code> (3.2).
diner_group_id FK	int	sim	aplicação	Aponta para <code>meal_diner_group.id</code> com <code>ON DELETE NO ACTION</code> . Obrigatória para <code>diner_type = 2</code> . O grupo é resolvido no servidor e conferido quanto a <code>is_active</code> antes de gravar.
host_company_code	varchar(10)	sim	aplicação	O anfitrião : quem convidou o visitante. As três colunas vêm juntas ou nenhuma vem (<code>CK_meal_log_host</code>) — “visitante convidado pela matrícula 000123 de empresa nenhuma” não permite rateio nem cobrança, é dado que parece dado. Não há FK: a chave é do Protheus. Quando o grupo tem <code>requires_host = 1</code> , servir sem anfitrião é recusado antes do banco.
host_employee_id	varchar(6)	sim	aplicação	
host_branch_code	varchar(10)	sim	aplicação	
guest_label	varchar(80)	sim	aplicação	Texto livre para nomear o visitante. Aceito na requisição (<code>trim</code> , vazio vira <code>NULL</code>); hoje é NULL nas 27 linhas — a tela não coleta o campo.
cost_center	varchar(9)	sim	servidor	Snapshot do <code>RA_CC</code> resolvido pela view no momento da gravação. Nunca aceito do cliente — aceitar permitiria rateio inventado. Para refeição de convidado, recebe o centro de custo do anfitrião : é o que faz o almoço do fornecedor virar custo do comprador em vez de custo sem dono. Tamanho 9 = <code>RA_CC</code> .
site_code	varchar(10)	não	aplicação	A loja onde a refeição foi servida . <code>CK_meal_log_site_code</code> exige o formato <code>[0-9][0-9][0-9][0-9]</code> . O formato é garantido; a <i>existência</i> da filial não pode ser (FK entre bancos não existe) e é conferida na abertura da sessão do operador, contra <code>SYS_COMPANY</code> .
served_at	datetime2(0)	não	aplicação · UTC	O instante em que a pessoa foi servida. Vem do relógio do aparelho quando a refeição foi registrada offline — não é fonte de verdade, mas é a melhor que existe nesse caso. A única correção aplicada é no futuro: um <code>served_at</code>

				adiantado criaria refeição em data que ainda não chegou, e o relatório do dia seguinte nasceria errado.
service_date	date	não	aplicação · local	A data civil do serviço, coluna real — nunca derivada de <code>served_at</code> na consulta. É o que garante que uma mudança futura na regra do corte de meia-noite não reescreva o passado. Primeira coluna dos dois índices de relatório. Hoje é sempre igual à data local de <code>served_at</code> (ver 14.2).
meal_type	tinyint	não	aplicação	1 almoço · 2 jantar · 3 café (<code>CK_meal_log_meal_type</code>).
identified_by	tinyint	não	aplicação	1 QR · 2 manual · 3 facial · 4 botão · 5 cupom. Para grupo é forçado a 4 pelo servidor, independentemente do que o cliente mandar.
match_score	decimal(5,4)	sim	aplicação	Similaridade de cosseno entre o rosto apresentado e o vetor guardado, na faixa 0–1 (ex.: <code>0.9222</code>). Só faz sentido com <code>identified_by = 3</code> , e a aplicação recusa a combinação contrária. Preenchido em 7 das 27 linhas — as que passaram por facial; os valores observados vão de <code>0.5370</code> a <code>0.9222</code> . O tipo aceita até <code>9.9999</code> , folga inofensiva.
operator_user_id	int	sim	aplicação	Id do usuário autenticado que registrou a refeição, vindo do JWT. Aponta para o cadastro de usuários no MySQL global , em outro servidor — logo, sem FK. Nas 27 linhas é sempre <code>148</code> .
client_uuid	UQ <code>uniqueidentifier</code>	não	aparelho	Gerado pelo aparelho , antes de qualquer rede. É a chave de idempotência da fila offline: reenviar o mesmo uuid não duplica a refeição e não devolve erro. Único por <code>UX_meal_log_client</code> .
synced_at	<code>datetime2(0)</code>	sim	aplicação · UTC	<code>NULL</code> quando a refeição foi registrada online; preenchida com o instante do descarregamento quando veio por <code>POST /logs/sync</code> . É o que permite ao relatório diário informar quantas refeições vieram da fila (coluna <code>veio_da_fila</code>). NULL nas 27 linhas : a fila offline nunca descarregou em produção — ver 14.11.

CONSTRAINTS

NOME	PREDICADO E PROPÓSITO
PK_meal_log	PRIMARY KEY CLUSTERED (id) .
CK_meal_log_diner	Os três ramos mutuamente exclusivos de <code>diner_type</code> (seção 4). É a constraint mais importante da tabela: impede meia identificação.
CK_meal_log_host	As três colunas <code>host_*</code> todas nulas, ou todas preenchidas.
CK_meal_log_meal_type	<code>meal_type IN (1,2,3)</code> .
CK_meal_log_identified_by	<code>identified_by IN (1,2,3,4,5)</code> . Alargado de (1..4) para (1..5) no deploy de 27/08/2026.
CK_meal_log_site_code	<code>site_code LIKE '[0-9][0-9][0-9][0-9]'</code> .
FK_meal_log_diner_group	<code>diner_group_id → meal_diner_group(id)</code> , NO ACTION nos dois lados.

ÍNDICES

ÍNDICE	CHAVE / INCLUÍDAS	CONSULTA QUE ELE SERVE
--------	-------------------	------------------------

PK_meal_log clustered	id	Ordem física; alvo da FK do cupom.
UX_meal_log_client único	client_uuid	Releitura por uuid e garantia final contra reenvio duplicado.
IX_meal_log_dup	company_code, employee_id, branch_code, service_date	“Quantas refeições esta pessoa fez hoje?” — pergunta feita a cada crachá lido, na fila.
IX_meal_log_rpt	service_date, site_code inclui cost_center, company_code	Relatórios por período e loja; service_date vem primeiro justamente porque todo relatório recorta por data.

EXEMPLO DE LEITURA — O QUE AS 27 LINHAS DE HOJE MOSTRAM

ID	LINHA	COMO SE LÊ
77	tipo 1 · 02/001981/0209 · cc 19907 · loja 0209 · almoço · facial · score 0.9222	Colaborador identificado por rosto, com folga sobre o limiar de 0,50.
78	tipo 2 · grupo 13 (Visitante) · host 02/001981/0209 · cc 19907 · botão	Visitante servido por botão, lançado no centro de custo do anfitrião 001981.
79	tipo 2 · grupo 15 (Jovem Aprendiz) · sem host · cc NULL · botão	Grupo que não exige anfitrião: sem host, e portanto sem centro de custo — refeição por conta da casa.
99	tipo 1 · 02/001292/0209 · loja 0202 · café · QR	Pessoa lotada em 0209 tomando café da manhã na loja 0202. É o caso que justifica as duas colunas separadas.
100	tipo 1 · 02/001292/0209 · loja 0202 · almoço · facial · score 0.5370	Reconhecimento aceito com 0,037 de folga sobre o limiar de 0,50. É o tipo de linha que a calibração do piloto tem que explicar — ver 14.11.

6. meal_diner_group — os grupos sem matrícula

O cadastro dos “baldes”: categorias de pessoas que são **contadas sem serem identificadas**. Cada linha ativa é um botão na tela do operador. É a única tabela do módulo com colunas de auditoria de alteração.

COLUNA	TIPO	NULO	DEFAULT	SIGNIFICADO E REGRA
id PK	int	não	IDENTITY(1,1)	Referenciado por meal_log.diner_group_id . Os ids em uso são 13, 15 e 16 — não sequenciais, resíduo de tentativas anteriores (14.7).
label	varchar(60)	não	—	O texto do botão. Editável por PATCH , porque é ajuste de tela. Valores atuais: Visitante, Jovem Aprendiz, Seguranca.
payee_id FK	int	sim	—	A contratante a quem a refeição é faturada — gipp_payee(id) , NO ACTION . Editável, porque contrato muda de contratante. NULL nas três linhas: nada é faturado hoje.
billable	bit	não	0	Separa o que se cobra do que só se conta. Visitante entra na contagem e não gera fatura. As três linhas estão em 0 .
requires_host	bit	não	0	Ligado, o grupo só pode ser servido depois do crachá de quem convidou. É assim que a refeição do visitante cai no centro de custo do comprador em vez de virar custo sem dono. Hoje só Visitante (id 13) tem 1 — e de fato todas as linhas de meal_log com grupo 13 têm host_* preenchido.
site_code	varchar(10)	sim	—	NULL significa “vale em todas as lojas”, e é o valor das três linhas. Quando preenchido, restringe o botão a uma filial. CK_meal_diner_group_site_code aceita NULL ou exatamente 4 dígitos.
sort_order	smallint	não	0	Ordem dos botões na tela (ORDER BY sort_order, label). Hoje 10 / 20 / 30, espaçados para permitir inserção no meio sem renumerar.
is_active	bit	não	1	Exclusão lógica. Desativar esconde o botão sem apagar o cadastro — e sem quebrar as refeições já registradas com aquele grupo, que é justamente o motivo do NO ACTION na FK.
created_by	int	não	—	Usuário que criou (MySQL global , sem FK). É 0 nas três linhas: elas nasceram do script de carga, não de um usuário real (14.8).
created_by_branch_code	varchar(10)	não	—	Filial do usuário no momento da criação — 0209 nas três.
created_at	datetime2(0)	não	SYSDATETIME() · local	18/08/2026 nas três linhas.
updated_by	int	sim	—	Preenchidas pelo PATCH , que escreve as três de uma vez.
updated_by_branch_code	varchar(10)	sim	—	NULL nas três linhas: nenhum grupo foi editado desde a criação. Os campos aceitos no PATCH são label , payee_id , billable , requires_host , site_code , sort_order e is_active .
updated_at	datetime2(0)	sim	SYSDATETIME() no UPDATE · local	

Índice: só **PK_meal_diner_group**. A tabela tem 3 linhas e é lida inteira a cada abertura de tela — índice adicional aqui seria custo sem retorno.

7. meal_biometric — o vetor facial e o consentimento

Um vetor por pessoa, mais a prova de que ela consentiu. **Não existe coluna de imagem:** as fotos capturadas no cadastro são convertidas em vetor em memória e descartadas sem tocar disco. A promessa “só o vetor” é restrição de schema, não disciplina de programador.

Dado pessoal sensível — art. 11, I da Lei 13.709/2018

O vetor **não** é tratado como anonimizado: a irreversibilidade de *embeddings* não é garantida pela literatura, e a classificação como sensível vem da finalidade — identificar unicamente uma pessoa — não do formato do arquivo. Toda esta tabela está sob o documento `docs/refeitorio-lgpd-tratamento-biometrico.md`, cuja seção 8 registra pendência aberta: a empresa não tem encarregado (DPO) nomeado.

COLUNA	TIPO	NULO	ESCRITO POR	SIGNIFICADO E REGRA
<code>company_code</code> PK	<code>varchar(10)</code>	não	aplicação	A PK é a trinca do Protheus — não há id técnico. Consequência de desenho: uma pessoa tem no máximo um vetor, e recadastrar é <code>MERGE</code> , não nova linha. Não sobra histórico de vetores antigos, o que é intencional: guardar vetores substituídos seria guardar biometria além da necessidade.
<code>employee_id</code> PK	<code>varchar(6)</code>	não	aplicação	
<code>branch_code</code> PK	<code>varchar(10)</code>	não	aplicação	
<code>embedding</code>	<code>varbinary(2048)</code>	não	container face	512 números float32 little-endian , exatamente 2048 bytes — e <code>CK_meal_biometric_embedding_size</code> exige <code>DATALENGTH = 2048</code> , então vetor de outro tamanho não entra. O vetor é normalizado (norma 1) antes de gravar, para que o produto escalar entre dois vetores <i>seja</i> a similaridade de cosseno; vetores com normas diferentes no banco produziram scores incomparáveis. Sai desta tabela apenas para o container de reconhecimento, na rede interna — nunca para fora do servidor, e nunca em resposta de API.
<code>model_tag</code>	<code>varchar(40)</code>	não	container face	Qual modelo gerou o vetor: <code>insightface-buffalo_l-w600k-r50</code> nas 5 linhas. Obrigatória, e usada como filtro na identificação 1:N — comparar vetores de modelos diferentes é comparar coisas não comparáveis. Trocar de modelo exige recadastrar, e esta coluna é o que torna a migração detectável em vez de silenciosa.
<code>enrolled_at</code>	<code>datetime2(0)</code>	não	aplicação · UTC DEFAULT existe, não é usado	Quando o vetor foi gravado. Vem por parâmetro, do mesmo <code>new Date()</code> de <code>consent_at</code> — ver 3.4.1.
<code>enroll_verified_by</code>	<code>tinyint</code>	não	aplicação	Como a identidade foi provada: 1 link assinado · 2 presencial · 3 primeiro QR. É a coluna de auditoria de cadastro suspeito. <code>CK_meal_biometric_verified_by</code> restringe a (1,2,3). As 5 linhas são todas 2.
<code>consent_at</code>	<code>datetime2(0)</code>	não	aplicação · UTC	Instante do consentimento. <code>CK_meal_biometric_consent_before_enroll</code> exige <code>consent_at <= enrolled_at</code> : não se grava biometria antes de consentir. Nas 5 linhas <code>consent_at = enrolled_at</code> , porque o fluxo presencial grava as duas do mesmo relógio.
<code>consent_version</code>	<code>varchar(20)</code>	não	aplicação	Versão do texto aceito — <code>refeitorio-facial-v1</code> nas 5 linhas, configurável por <code>MEAL_CONSENT_VERSION</code> . É o que

				permite saber exatamente ao que cada pessoa aderiu quando o texto mudar. Sem esta coluna, mudar o termo apagaria o significado de todo consentimento anterior.
consent_ip	varchar(45)	sim	aplicação	IP de origem do consentimento; 45 caracteres cabem IPv6. <code>10.10.10.99</code> nas 5 linhas — cadastro feito por aparelho na rede interna.
revoked_at	datetime2(0)	sim	aplicação · UTC	Revogação do consentimento. Marca, não apaga: o expurgo leva na execução seguinte, e fica rastro auditável de que a revogação foi respeitada em vez de a linha simplesmente desaparecer. <code>revoked_at IS NULL</code> está no <code>WHERE</code> de toda consulta que lê vetor — não em quem chama —, porque esquecer esse filtro numa camada acima seria ignorar uma revogação. <code>CK_meal_biometric_revoked_after_enroll</code> exige <code>revoked_at >= enrolled_at</code> ; recadastrar depois de revogar zera esta coluna e reativa a pessoa. <code>NULL</code> nas 5 linhas.

ÍNDICES

ÍNDICE	CHAVE / INCLUÍDAS	CONSULTA QUE ELE SERVE
<code>PK_meal_biometric</code> clustered	<code>company_code</code> , <code>employee_id</code> , <code>branch_code</code>	Comparação 1:1: a matrícula estreita para uma linha e o rosto só confirma.
<code>IX_meal_biometric_revoked</code>	<code>revoked_at</code> inclui <code>company_code</code> , <code>employee_id</code> , <code>branch_code</code> , <code>model_tag</code>	Varredura de candidatos ativos (identificação 1:N) e listagem do que o expurgo precisa levar — sem tocar na tabela base, ou seja, sem ler o vetor .

8. meal_enroll_token — o convite de autocadastro

Estado do link individual que o RH emite para alguém se cadastrar no facial. Existe porque adesão voluntária em escala precisa de um caminho que não passe pelo balcão — e porque cada tentativa frustrada precisa ser contada, não repetida à vontade.

COLUNA	TIPO	NULO	ESCRITO POR	SIGNIFICADO E REGRA
<code>jti</code> PK	<code>uniqueidentifier</code>	não	aplicação	O <i>JWT ID</i> : o mesmo identificador que vai assinado dentro do link enviado à pessoa. A PK ser o <code>jti</code> é o que amarra token emitido e linha de estado — não há como validar um link cujo estado não exista.
<code>company_code</code>	<code>varchar(10)</code>	não	aplicação	Para quem o convite foi emitido. A trinca é a chave do índice único filtrado que impede dois convites vivos para a mesma pessoa.
<code>employee_id</code>	<code>varchar(6)</code>	não	aplicação	
<code>branch_code</code>	<code>varchar(10)</code>	não	aplicação	
<code>issued_at</code>	<code>datetime2(0)</code>	não	<code>SYSDATETIME()</code> · local	Emissão. É a única coluna de data desta tabela que não vem da aplicação — e daí a mistura de fusos descrita em 14.1.
<code>expires_at</code>	<code>datetime2(0)</code>	não	aplicação · UTC	Validade. Calculada como “agora + <code>MEAL_ENROLL_TTL_DAYS</code> ” (7 dias por padrão). <code>CK_meal_enroll_token_expires</code> exige <code>expires_at > issued_at</code> .
<code>issued_by</code>	<code>int</code>	não	aplicação	Usuário do RH que emitiu (MySQL <code>global</code> , sem FK). 148 nas 6 linhas.
<code>attempts</code>	<code>tinyint</code>	não	aplicação	Tentativas de conferência de data de nascimento já gastas. <code>CK_meal_enroll_token_attempts</code> limita a 0–3, e o incremento só ocorre com <code>attempts < 3</code> — na mesma instrução da leitura, via <code>OUTPUT</code> , para que duas tentativas simultâneas não gastem uma só.
<code>consumed_at</code>	<code>datetime2(0)</code>	sim	<code>SYSDATETIME()</code> · local	Convite usado com sucesso — gravado na mesma transação do vetor. As duas coisas juntas ou nenhuma: token consumido sem vetor gravado deixaria a pessoa sem cadastro e sem link, isto é, sem caminho de volta a não ser suporte manual.
<code>burned_at</code>	<code>datetime2(0)</code>	sim	<code>SYSDATETIME()</code> · local	Convite invalidado sem ter sido usado. <code>CK_meal_enroll_token_outcome</code> impede que <code>consumed_at</code> e <code>burned_at</code> coexistam: um convite tem um único desfecho.
<code>burned_reason</code>	<code>varchar(40)</code>	sim	aplicação	Por que foi queimado — <code>reissued</code> , <code>birth_date_missing</code> ou <code>attempts_exhausted</code> . <code>CK_meal_enroll_token_burn_reason</code> exige que <code>burned_at</code> e <code>burned_reason</code> sejam nulos juntos ou preenchidos juntos: não existe queima sem motivo registrado.

ESTADOS POSSÍVEIS DE UMA LINHA

ESTADO	COMO SE RECONHECE	LINHAS HOJE (6)
--------	-------------------	-----------------

Aberto	<code>consumed_at IS NULL AND burned_at IS NULL</code>	Nenhuma. É este predicado que forma o índice único filtrado — no máximo um aberto por pessoa.
Consumido	<code>consumed_at IS NOT NULL</code>	2 — uma com <code>attempts = 2</code> , outra com <code>attempts = 3</code> : acertaram na última tentativa disponível.
Queimado	<code>burned_at IS NOT NULL</code>	4, todas com motivo <code>reissued</code> e <code>attempts = 0</code> : o RH reemitiu antes de a pessoa tentar.
Expirado	aberto, e <code>expires_at <= SYSDATETIME()</code>	Não é estado gravado: é calculado na leitura, na própria consulta que valida o link.

As 6 linhas são todas da matrícula `02/001981/0209`, em 19/08/2026 — a sequência de reemissões do teste do fluxo. Índices: `PK_meal_enroll_token ( jti )` e `UX_meal_enroll_token_open`, único e filtrado por `consumed_at IS NULL AND burned_at IS NULL`.

Um dado que esta tabela deliberadamente não guarda

A conferência de identidade compara a data de nascimento digitada com o `RA_NASC` do Protheus, lido por `OUTER APPLY` na hora da validação. **Esse `RA_NASC` não é gravado em lugar nenhum** — nem no token, nem em log: existe em memória pelo tempo de uma comparação. E o nome da pessoa só é devolvido pela API *depois* de a data bater; devolver antes transformaria o endpoint num oráculo — manda matrícula, recebe nome.

9. meal_coupon — o saldo do cupom fiscal

Venda de almoço a prestador de serviço, paga no caixa e comprovada por NFC-e. A tabela é **controle de saldo**, não lista de cupons usados: um cupom que libera 4 almoços vira 4 linhas, com `seq` de 1 a 4. Criada em 27/08/2026 e ainda **vazia** — nenhum resgate em produção.

COLUNA	TIPO	NULO	ESCRITO POR	SIGNIFICADO E REGRA
<code>id</code> PK	<code>bigint</code>	não	IDENTITY(1,1)	Chave técnica de cada refeição consumida do cupom.
<code>nfe_key</code>	<code>char(44)</code>	não	aplicação	Chave de acesso da NFC-e, 44 dígitos. <code>CK_meal_coupon_nfe_key</code> exige <code>NOT LIKE '%[^0-9]%'</code> — só dígitos. Não é dado pessoal : identifica uma venda, não um comprador — e é justamente por isso que este caminho não precisa de matrícula nem de nome.
<code>seq</code>	<code>smallint</code>	não	aplicação	O contador de saldo, de 1 a <code>meals_authorized</code> . Calculado como <code>ISNULL(MAX(seq),0)+1</code> sob <code>UPDLOCK, HOLDLOCK</code> .
<code>meals_authorized</code>	<code>smallint</code>	não	aplicação	Quantidade do item de almoço no cupom, lida de <code>MFL_DFITEM</code> (Consinco) no primeiro resgate e repetida nas linhas seguintes. Gravada e não recalculada: se o Consinco mudar depois, o saldo já concedido não pode encolher com gente esperando na fila. <code>CK_meal_coupon_authorized</code> exige <code>>= 1</code> .
<code>site_code</code>	<code>char(4)</code>	não	aplicação	Loja onde o almoço foi servido — igual à do cupom por definição, porque a validação recusa antes se divergirem. Note que aqui o tipo é <code>char(4)</code> , mais estrito que o <code>varchar(10)</code> de <code>meal_log</code> .
<code>coupon_date</code>	<code>date</code>	não	aplicação	<code>DTAMOVIMENTO</code> do Consinco: o dia da compra .
<code>service_date</code>	<code>date</code>	não	aplicação	O dia do serviço . Hoje sempre igual a <code>coupon_date</code> , porque a regra é “só cupom do dia”. As colunas são separadas porque a regra pode afrouxar — cupom da véspera para o café da manhã — e aí o passado não pode ser reinterpretado. Mesma razão de <code>service_date</code> existir em <code>meal_log</code> .
<code>seqproduto</code>	<code>int</code>	não	aplicação	Qual produto liberou a refeição. Gravado porque o código do almoço muda com o tempo, e o relatório de seis meses atrás precisa continuar explicável.
<code>tp_emis</code>	<code>char(1)</code>	não	aplicação	<code>1</code> normal, <code>9</code> contingência (<code>CK_meal_coupon_tp_emis</code>). Contingência é aceita quando o documento está no Consinco, mas fica marcada — a venda ainda pode ser rejeitada pela SEFAZ, e o relatório de conferência precisa achar essas linhas.
<code>meal_log_id</code> FK	<code>bigint</code>	sim	aplicação	Amarra a linha de saldo à refeição correspondente em <code>meal_log</code> , depois que esta ganhou <code>id</code> . <code>NO ACTION</code> . É nula na janela entre o débito e a gravação da refeição, dentro da mesma transação.
<code>operator_user_id</code>	<code>int</code>	sim	aplicação	Operador que fez o resgate (MySQL <code>global</code> , sem FK).
<code>redeemed_at</code>	<code>datetime2(0)</code>	não	aplicação · UTC <code>DEFAULT SYSDATETIME()</code>	Instante do resgate. Vem por parâmetro justamente para casar com <code>meal_log.served_at</code> : o <code>DEFAULT</code> usava o relógio local do SQL Server enquanto <code>served_at</code> vem em UTC, e o mesmo resgate ficava gravado com três horas de diferença entre as duas tabelas. Um relógio só para o par.

ÍNDICES E O MECANISMO DO SALDO

ÍNDICE	CHAVE / INCLUÍDAS	CONSULTA QUE ELE SERVE
PK_meal_coupon clustered	id	—
UX_meal_coupon_seq único	nfe_key, seq	A garantia final do saldo: dois terminais no mesmo QR, um passa e o outro recalcula.
IX_meal_coupon_key	nfe_key inclui seq	A consulta quente — “quanto sobrou deste cupom?” — uma vez por leitura de QR, na fila. seq incluído para o MAX sair do índice sem tocar na tabela.
IX_meal_coupon_rpt	service_date, site_code inclui nfe_key, tp_emis, meals_authorized	Relatório de conferência: cupons do dia, e os de contingência que ainda precisam ser cruzados com o que a SEFAZ autorizou.

Por que o cupom não passa pela rota normal de refeição

Debitar o saldo e gravar a refeição são indivisíveis, e a rota `POST /logs` não tem transação — pior, ela alimenta a fila de `POST /logs/sync`, que é exatamente onde o mesmo cupom passaria duas vezes. Por isso `diner_type = 3` é **recusado** por `validateMealLog`, com mensagem apontando para `POST /gipp/meal/coupons/redeem`, que debita e grava na mesma transação. A recusa é estrutural, não esquecimento.

10. vw_meal_diner — quem pode ser servido

A view que resolve as 7 empresas do Protheus em **uma linha por pessoa**. É a fonte de verdade do comensal, e a razão pela qual não existe tabela de colaboradores neste módulo.

COMO É MONTADA

- **UNION ALL de 7 ramos** — SRA010, SRA020, SRA030, SRA060, SRA070, SRA080, SRA090, cada um carimbando seu `company_code` literal e filtrando `D_E_L_E_T_ <> '*'`.
- **OUTER APPLY com TOP 1**, não `LEFT JOIN`, para descrição do centro de custo e nome da empresa. Motivo: `CTT` tem chave `CTT_FILIAL + CTT_CUSTO`, e juntar só por `CTT_CUSTO` duplicaria o colaborador quando o mesmo centro de custo existe em mais de uma filial. Numa view que alimenta cache offline, linha duplicada é a mesma pessoa aparecendo duas vezes na tela do operador. O `ORDER BY CTT_FILIAL` torna a escolha determinística.
- **branch_code vem de RA_FILIAL**, direto da tabela da pessoa — não do `SYS_COMPANY` por `LEFT JOIN`. Filial ausente ou deletada no cadastro produziria `branch_code` nulo, e a pessoa perderia metade da chave.
- **Não expõe CPF**. O refeitório não precisa de CPF para servir almoço (princípio da necessidade, art. 6º, III), e o que não está na view não vaza por descuido de quem escrever a próxima consulta.
- **Desligado não é filtrado**. `is_terminated` e `terminated_at` existem para a tela acender o alerta. Quem decide a janela do cache é a aplicação — política não mora em view.

COLUNA	ORIGEM	SIGNIFICADO
<code>company_code</code>	literal do ramo	Sufixo da SRAxxx : 01 ... 09 .
<code>employee_id</code>	RA_MAT	Matrícula, com <code>LTRIM/RTRIM</code> na projeção — o filtro cai sobre as colunas cruas nos ramos do <code>UNION</code> , para não perder a sargabilidade.
<code>branch_code</code>	RA_FILIAL	Lotação da pessoa.
<code>employee_name</code>	RA_NOME	Nome curto — o que aparece na tela do operador.
<code>employee_full_name</code>	RA_NOMECMP	Nome completo.
<code>cost_center</code>	RA_CC	Vazio vira <code>NULL</code> (<code>NULLIF</code>). É o valor copiado para <code>meal_log.cost_center</code> .
<code>cost_center_description</code>	CTT_DESC01	Rótulo, buscado por <code>OUTER APPLY TOP 1</code> .
<code>branch_name</code>	M0_FILIAL	Nome da filial.
<code>company_name</code>	M0_NOME	Razão social.
<code>is_terminated</code>	derivado	1 quando <code>RA_DEMISSA</code> não está vazio.
<code>terminated_at</code>	RA_DEMISSA	<code>TRY_CONVERT(DATE, ..., 112)</code> — formato <code>YYYYMMDD</code> do Protheus. Usada na comparação que importa: <code>terminated_at <= service_date</code> aponta quem já estava desligado no dia , em vez de marcar como irregular a refeição de agosto de quem saiu em setembro.
<code>admitted_at</code>	RA_ADMISSA	Idem, para admissão.

Cardinalidade garantida: 1 linha por (`company_code`, `employee_id`, `branch_code`). Preço aceito do `TOP 1` : quando o mesmo centro de custo existe em várias filiais, a descrição escolhida é a da primeira filial em ordem — aceitável para campo de exibição.

11. Objetos externos referenciados sem chave estrangeira

Metade do modelo aponta para fora. Estas são as referências que **não** têm FK, com o que substitui a garantia em cada caso — porque “não tem FK” não significa “não tem controle”.

OBJETO	ONDE VIVE	QUEM APONTA	O QUE SUBSTITUI A FK
<code>gipp_payee</code>	<code>GIPP.dbo</code> mesmo banco	<code>meal_diner_group.payee_id</code>	Tem FK — é a única referência externa ao módulo com integridade declarada. Colunas usadas nos relatórios: <code>id</code> , <code>name</code> , <code>document</code> .
<code>SRA010</code> ... <code>SRA090</code>	<code>TMPPRD12.dbo</code>	a trinca do comensal, em 3 tabelas	Resolução obrigatória pela view antes de gravar: matrícula que não resolve devolve <code>DINER_NOT_FOUND</code> , e a refeição não é registrada.
<code>CTT010</code> ... <code>CTT090</code>	<code>TMPPRD12.dbo</code>	<code>meal_log.cost_center</code>	Nada — é snapshot, e não precisa continuar existindo para o histórico permanecer legível.
<code>SYS_COMPANY</code>	<code>TMPPRD12.dbo</code>	<code>site_code</code> , <code>branch_code</code>	<code>CHECK</code> de formato (4 dígitos) + conferência de existência na abertura da sessão do operador, não a cada refeição.
<code>MFL_DFITEM</code> , <code>GE_EMPRESA</code>	<code>CONSINCO</code>	<code>meal_coupon.nfe_key</code> , <code>seqproduto</code> , <code>meals_authorized</code>	O resgate só grava depois de o documento e o item existirem no Consinco. A tradução CNPJ → loja sai de <code>SYS_COMPANY</code> ; CNPJ → <code>NROEMPRESA</code> , da própria consulta do cupom.
Usuários e permissões	MySQL <code>global</code> outro servidor	<code>operator_user_id</code> , <code>issued_by</code> , <code>created_by</code> , <code>updated_by</code>	Autenticação por JWT antes da rota. As permissões que governam o módulo: <code>MEAL_SERVE</code> , <code>MEAL_MANAGE_GROUPS</code> , <code>MEAL_VIEW_REPORT</code> , <code>MEAL_MANAGE</code> .
Container <code>face</code>	rede interna loopback	<code>embedding</code> , <code>model_tag</code> , <code>match_score</code>	Nenhuma integridade referencial a garantir; o que importa é que o vetor não sai da rede da empresa — não há envio para API de terceiro.

12. Fluxos — que linhas nascem em cada operação

12.1 Refeição de colaborador, online

1. Operador lê o QR (ou digita, ou usa o rosto). A aplicação resolve a matrícula em `vw_meal_diner` e traz, na **mesma ida ao banco**, quantas refeições a pessoa já fez hoje — a tela mostra nome + contagem, e duas requisições numa fila de 300 pessoas é latência evitável.
2. Duplicata **sinaliza, não bloqueia**: `meals_today > 0` vira alerta na tela; a decisão é do operador.
3. `validateMealLog` confere o payload; o servidor decide `cost_center` (snapshot da view) e `service_date` (data civil de `served_at`) — nenhum dos dois é aceito do cliente.
4. **1 linha em `meal_log`**, via `INSERT ... WHERE NOT EXISTS (client_uuid)`, e um `SELECT` no fim devolve a linha — a nova ou a que já estava lá.

12.2 Refeição registrada offline e sincronizada depois

1. O aparelho serve com o cache local de comensais e gera o `client_uuid` **antes de qualquer rede**.
2. `POST /logs/sync` descarrega a fila. Cada item passa pelo mesmo caminho de 12.1, com `synced_at` preenchido.
3. Reenvio do mesmo `client_uuid` não duplica: o `WHERE NOT EXISTS` cobre o caso normal e `UX_meal_log_client` resolve a corrida entre dois envios simultâneos (o repositório reconsulta e devolve a linha existente).
4. `served_at` vem do relógio do aparelho, com correção só para o futuro; `cost_center` é o do momento do sincronismo.

12.3 Refeição de grupo (balde) e de visitante

1. O operador toca o botão do grupo. O servidor lê o grupo por id, confere `is_active` e força `identified_by = 4`.
2. Se `requires_host = 1` e não vier `host_employee_id`, a operação é recusada com mensagem para passar o crachá do anfitrião.
3. Com anfitrião, a aplicação resolve **a matrícula dele** na view e copia **o centro de custo dele** para a refeição. Anfitrião desligado gera alerta, não bloqueio.
4. **1 linha em `meal_log`** com `diner_type = 2`, `diner_group_id` preenchido e a trinca do colaborador nula.

12.4 Autocadastro facial por link

1. RH emite: **1 linha em `meal_enroll_token`**. Se já existir convite aberto para a pessoa, o índice único filtrado recusa — reemitir exige queimar o anterior (`burned_reason = 'reissued'`).
2. A pessoa abre o link e digita a data de nascimento. Cada tentativa gasta um `attempts` (`UPDATE ... OUTPUT`); na terceira falha o convite é queimado com `attempts_exhausted`.
3. Data conferida, a API devolve o nome para a pessoa confirmar, e ela lê e aceita o termo.
4. A imagem vai ao container `face`, que devolve 512 `float32` normalizados. **A imagem não toca disco.**
5. **Na mesma transação**: `MERGE` em `meal_biometric` (insere ou substitui o vetor, e zera `revoked_at`) e `UPDATE` de `consumed_at` no token. As duas coisas juntas ou nenhuma.

O caminho presencial (`enrollDirect`) dispensa o token e grava `enroll_verified_by = 2`. O consentimento continua sendo **da pessoa**: quem toca “concordo” é ela, no aparelho, depois de ler o termo — o operador só passa o aparelho.

12.5 Identificação por rosto na fila

1. Recorte por loja: candidatos são quem é lotado na filial **ou** quem comeu ali nos últimos 30 dias. Quem não cai em nenhum dos dois usa o crachá — um visitante eventual não deve estar no conjunto de busca de uma loja onde nunca

apareceu.

2. Filtro por `model_tag` e por `revoked_at IS NULL`, ambos no `WHERE` da consulta.
3. Duas travas para aceitar: similaridade acima do limiar (`MEAL_FACE_THRESHOLD`, hoje 0,50) e margem mínima sobre o segundo colocado (`MEAL_FACE_MARGIN`, hoje 0,06). Empate devolve `ambiguous` e a tela pede o crachá — irmãos e gêmeos são o caso comum, não o exótico, numa base de mil e setecentas pessoas.
4. Aceito, segue 12.1 com `identified_by = 3` e `match_score` preenchido.

Os dois números são provisórios e são o entregável do piloto. Limiar não sai de *paper*: depende da câmera do aparelho, da luz do refeitório e das fotos da própria empresa. O que a etapa 6 deve entregar não é uma tela funcionando — é esse par de números, medido com 15 a 20 voluntários, com taxa de falso aceite e falso rejeite em vários cortes.

12.6 Resgate de cupom fiscal

1. Leitura da chave de 44 dígitos da NFC-e. CNPJ → loja por `SYS_COMPANY`; nada encontrado é `unknown-store` (cupom de outra empresa).
2. Loja do cupom comparada com a loja da sessão; divergência recusa.
3. Consinco valida o documento e devolve a quantidade do item de almoço.
4. Saldo lido de `meal_coupon` por `nfe_key`; `meals_authorized` é o da **primeira** linha, não o maior.
5. Numa transação: **1 linha em `meal_coupon`** (com `seq` calculado sob `UPDLOCK`, `HOLDLOCK`), **1 linha em `meal_log`** com `diner_type = 3` e `identified_by = 5`, e o `UPDATE` que grava `meal_log_id` na linha do cupom.

13. Ciclo de vida, retenção e LGPD

DADO	RETENÇÃO	COMO TERMINA
<code>meal_log</code>	Indefinida	Não é apagado. Matrícula, data e loja não são dado sensível, e o registro é documento de custo e de auditoria. <i>Some o vetor, fica a contabilidade.</i>
<code>meal_biometric.embedding</code>	30 dias após o desligamento, ou a revogação — o que vier primeiro	Consulta de expurgo agendada, versionada junto do código. O vetor mora no mesmo banco do cadastro de pessoal justamente por causa dela: é um <code>JOIN</code> na mesma instância. Em bancos separados, o expurgo dependeria de um job de sincronismo funcionar — e isso não é frase para dizer à ANPD. Ver 14.4: a consulta publicada não cobre o que promete.
Imagem facial	Nenhuma	Convertida em vetor em memória e descartada sem tocar disco. Não existe coluna onde guardar.
<code>RA_NASC</code> na conferência	Nenhuma	Lido por <code>OUTER APPLY</code> no instante da comparação; não é gravado nem logado.
<code>meal_enroll_token</code>	Sem política definida	Nenhuma rotina apaga tokens consumidos ou queimados. O volume é irrisório; a linha é rastro de auditoria de emissão.
<code>meal_coupon</code>	Indefinida	É saldo e é documento fiscal — não se apaga.

ONDE CADA PROMESSA DO DOCUMENTO DE LGPD APARECE NO SCHEMA

PROMESSA	GARANTIA
Só o vetor, nunca imagem	Ausência de coluna de imagem em <code>meal_biometric</code> .
Modelo identificado	<code>model_tag</code> , obrigatória.
Consentimento datado e versionado	<code>consent_at</code> , <code>consent_version</code> , <code>consent_ip</code> , com <code>CHECK</code> de ordem contra <code>enrolled_at</code> .
Revogação respeitada	<code>revoked_at</code> + <code>revoked_at IS NULL</code> no <code>WHERE</code> de toda leitura de vetor.
A alternativa ao facial existe e é usada	<code>identified_by</code> em cada refeição.
Auditoria de cadastro suspeito	<code>enroll_verified_by</code> .
Vetor não sai da rede	Container publicado só em <i>loopback</i> .

14. Observações técnicas e divergências encontradas

Achados da leitura cruzada entre catálogo, código e documentação. Não são defeitos hipotéticos: cada item aponta o objeto e o que se observa nele. A ordem é por consequência, não por dificuldade de correção.

14.1 O convite vale 7 dias e 3 horas, não 7 dias

`issued_at` é gravado por `DEFAULT SYSDATETIME()` (hora local, UTC-3) e `expires_at` pela aplicação (UTC). A validade é calculada como “agora + 7 dias” em UTC, e a expiração é conferida com `expires_at <= SYSDATETIME()` — ou seja, **UTC comparado com local**. Evidência nas 6 linhas: `issued_at 2026-08-19 17:36:02` → `expires_at 2026-08-26 20:36:47`, 7 dias e 3 horas depois.

Efeito: todo link vive 3 horas além do previsto, e `DATEDIFF` entre as duas colunas nunca dá o TTL configurado. **Correção mínima:** gravar `issued_at` pela aplicação, no mesmo `new Date()` de `expires_at` — exatamente o remédio já aplicado em `meal_biometric` em 19/08/2026 (3.4.1). Enquanto isso não acontece, é a diferença entre uma constraint que mede eventos e uma que mede relógios.

14.2 `served_at` em UTC, `service_date` em hora local

`service_date` é montada com as partes **locais** do instante (`getFullYear/getMonth/getDate`), enquanto `served_at` é gravado em UTC pelo driver. Entre 21:00 e 23:59 locais, `served_at` cai no **dia seguinte** ao de `service_date`.

Efeito: os relatórios estão certos, porque todos agrupam pela coluna `service_date` — que é justamente o motivo de ela existir. Mas as colunas de *horário* que o relatório de duplicatas devolve (`MIN/MAX(served_at)`) chegam em UTC: quem lê “primeira 12:30” está vendo 09:30 de Brasília. Com jantar registrado (`meal_type = 2`) o efeito passa a atingir a data também, se alguma consulta futura derivar dia a partir de `served_at`. **Correção:** converter na apresentação, ou padronizar as gravações em hora local.

14.3 A documentação de LGPD assinável promete 1:1; o código faz 1:N

`docs/refeitorio-lgpd-tratamento-biometrico.md`, seção 4, afirma: “Comparação 1:1, nunca 1:N. A matrícula estreita a busca para uma única linha; o rosto apenas confirma. Não existe função de *identificar esta pessoa entre todas* no sistema.” O código atual **tem** essa função: `sqlFindIdentifyCandidates` busca candidatos por loja e o caso de uso escolhe o melhor por limiar e margem — é 1:N recortado, com duas travas, mas 1:N.

Efeito: o documento que existe para ser assinado descreve um tratamento menos abrangente que o implementado. O desenho do recorte é defensável — a maior loja tem ~164 pessoas em vez das ~1.700 do grupo — mas essa defesa precisa estar **no documento**, e é ele que registra o risco aceito. **Ação:** atualizar a seção 4 antes da assinatura, descrevendo o recorte por loja, o limiar e a margem.

14.4 A consulta de expurgo publicada não entrega as duas promessas

A consulta da seção 4 do documento de LGPD faz `INNER JOIN` apenas com `TMPPRD12.dbo.SRA020`, e junta por `RA_MAT + RA_FILIAL` — **sem** `company_code`. Duas consequências:

- **Cobre uma empresa só.** Vetor de pessoa das outras seis (`01`, `03`, `06` – `09`) nunca é alcançado, e o prazo de 30 dias não se cumpre para ela. Hoje as 5 linhas são todas empresa `02`, então o problema é latente, não ativo.
- **Pode casar a pessoa errada.** Sem `company_code` na junção, matrícula igual em filial igual de outra empresa é a mesma linha para a consulta.

Além disso, a consulta **não trata revogação**: só apaga por desligamento. A promessa “o vetor é marcado como revogado e removido no expurgo seguinte” não é cumprida por ela — falta um segundo `DELETE` por `revoked_at IS NOT NULL`. **Ação:** reescrever o expurgo cobrindo as 7 empresas, com `company_code` na junção, e com o ramo de revogação.

14.5 meal_coupon está criada e nunca foi usada

Zero linhas. A estrutura, as constraints e os índices estão em produção desde 27/08/2026, e o `CHECK` de `meal_log` já aceita `diner_type = 3` — mas nenhum resgate ocorreu. O mecanismo de saldo (índice único + `CHECK` + `UPDLOCK`) **ainda não foi exercitado sob concorrência real**. Vale um teste com dois terminais no mesmo QR antes de considerar o caminho validado.

14.6 enroll_verified_by = 3 existe e não tem quem o grave

A constante `FIRST_MEAL_QR` está no domínio e o valor `3` é aceito pela constraint, mas nenhum caminho do código o escreve — só `1` (link) e `2` (presencial) são usados. Ou o fluxo “confirma no primeiro almoço por QR” está previsto e não foi implementado, ou o valor é resíduo. Nenhum dano; vale decidir qual dos dois.

14.7 Lacunas de id nas duas tabelas com IDENTITY

`meal_diner_group` tem ids 13, 15 e 16 — o 14 não existe. `meal_log` tem 27 linhas com ids de 68 a 92 e depois 99 e 100: os valores 93 a 98 foram consumidos pelo `IDENTITY` e não estão na tabela. Lacuna em `IDENTITY` **não prova exclusão** (transação revertida também consome o valor), mas `meal_log` é tabela append-only cuja premissa é que nada sai dela. Vale confirmar se foram *rollbacks* de teste ou `DELETE` manual — e, se houver `DELETE` manual acontecendo, isso conflita com o papel da tabela como documento de auditoria.

14.8 created_by = 0 não corresponde a usuário nenhum

As três linhas de `meal_diner_group` têm `created_by = 0` — nasceram do script de carga. A coluna é `NOT NULL`, sem *default* e sem FK (o cadastro está em outro banco), então `0` passa. Não é defeito de dados, é ausência de convenção: hoje não há como distinguir “criado pelo sistema” de “criado por um usuário cujo id não existe mais”. Um valor reservado documentado resolveria.

14.9 Tipos divergentes para o mesmo conceito

CONCEITO	TIPOS EM USO	CONSEQUÊNCIA
Código de loja / filial	<code>char(4)</code> em <code>meal_coupon</code> ; <code>varchar(10)</code> em <code>meal_log</code> e <code>meal_diner_group</code>	O <code>varchar(10)</code> aceita 10 caracteres onde só 4 são válidos; o formato é garantido por <code>CHECK</code> , não pelo tipo. Comparar as duas colunas funciona, mas é conversão implícita.
Autor de um registro	<code>int</code> em <code>meal_diner_group</code> ; <code>varchar(50)</code> em <code>gipp_payee</code>	Mesmo conceito, dois tipos, no mesmo banco. Qualquer relatório que una os dois precisa converter.

Nenhum dos dois é urgente. Anotados porque padronizar depois, com dados dentro, custa mais.

14.11 O que os dados dizem sobre o uso real (27 refeições)

MEDIDA	VALOR	LEITURA
Distribuição de <code>identified_by</code>	QR 10 · facial 7 · botão 10	Manual (2) e cupom (5) nunca foram usados . Os dois caminhos existem no código e no <code>CHECK</code> , e não têm nenhuma linha que os exercite.

Refeições vindas da fila	<code>synced_at</code> preenchido em 0	O caminho offline — a razão de existirem <code>client_uuid</code> , <code>synced_at</code> e o <code>INSERT ... WHERE NOT EXISTS</code> — nunca rodou em produção. É o mecanismo mais elaborado do módulo e o menos testado com dado real.
Menor <code>match_score</code> aceito	0.5370	0,037 acima do limiar de 0,50, na linha <code>id = 100</code> . Com o limiar atual, aceites rentes acontecem — e é exatamente o número que o piloto precisa medir com 15 a 20 voluntários antes de virar padrão.
Tipos de refeição	almoço 25 · café 2	Jantar (<code>meal_type = 2</code>) sem nenhuma linha.
Operadores distintos	1 (id 148)	Todas as 27 refeições, os 6 convites e nada mais: o módulo está em uso por uma única conta. Ainda é operação de teste, não rotina de refeitório.

Nenhum desses pontos é defeito. São a medida de **quanto do modelo já foi exercitado** — útil para decidir onde vale investir teste antes de escalar.

14.12 Pontos menores

- `meal_enroll_token` sem índice em `expires_at`. Qualquer rotina de limpeza por validade varre a tabela. Com 6 linhas, irrelevante — anotado para quando a adesão escalar.
- `meal_coupon.redeemed_at` mantém o `DEFAULT SYSDATETIME()` mesmo com a aplicação enviando o valor. Não há conflito no caminho normal, mas um `INSERT` manual gravaria hora local numa coluna que o resto da tabela tem em UTC.
- `meal_biometric` não registra quem revogou. Há `revoked_at`, e nada sobre canal ou autor da revogação — enquanto o cadastro tem `enroll_verified_by` e `consent_ip`. Se a revogação vier a ser feita pelo RH em nome do titular, a assimetria vira lacuna de auditoria.
- **Substituir o vetor não deixa histórico.** O `MERGE` sobrescreve `embedding`. É intencional (guardar vetor substituído seria guardar biometria além da necessidade), mas significa que não há como investigar “o cadastro de antes reconhecia melhor?”.

Anexo A — DDL reconstruído a partir do catálogo

Reconstrução fiel ao que está na instância em 28/08/2026: tipos, nulidade, *defaults*, constraints e índices lidos de `sys.*`. Serve para conferência e para recriar o modelo em ambiente de teste. **Não substitui os scripts de deploy originais** — a ordem de execução e as transações deles importam.

```
-- -- meal_diner_group -----
CREATE TABLE GIPP.dbo.meal_diner_group (
    id                INT IDENTITY(1,1) NOT NULL,
    label             VARCHAR(60)  NOT NULL,
    payee_id          INT          NULL,
    billable          BIT          NOT NULL CONSTRAINT DF_mdg_billable      DEFAULT (0),
    requires_host     BIT          NOT NULL CONSTRAINT DF_mdg_requires_host DEFAULT (0),
    site_code         VARCHAR(10)  NULL,
    sort_order        SMALLINT     NOT NULL CONSTRAINT DF_mdg_sort_order  DEFAULT (0),
    is_active         BIT          NOT NULL CONSTRAINT DF_mdg_is_active   DEFAULT (1),
    created_by        INT          NOT NULL,
    created_by_branch_code VARCHAR(10) NOT NULL,
    created_at        DATETIME2(0) NOT NULL CONSTRAINT DF_mdg_created_at  DEFAULT (SYSDATETIME()),
    updated_by        INT          NULL,
    updated_by_branch_code VARCHAR(10) NULL,
    updated_at        DATETIME2(0) NULL,
    CONSTRAINT PK_meal_diner_group PRIMARY KEY CLUSTERED (id),
    CONSTRAINT CK_meal_diner_group_site_code
        CHECK (site_code IS NULL OR site_code LIKE '[0-9][0-9][0-9][0-9]'),
    CONSTRAINT FK_meal_diner_group_payee
        FOREIGN KEY (payee_id) REFERENCES GIPP.dbo.gipp_payee (id)
);

-- -- meal_log -----
CREATE TABLE GIPP.dbo.meal_log (
    id                BIGINT IDENTITY(1,1) NOT NULL,
    diner_type        TINYINT          NOT NULL,
    company_code      VARCHAR(10)      NULL,
    employee_id       VARCHAR(6)       NULL,
    branch_code       VARCHAR(10)      NULL,
    diner_group_id    INT              NULL,
    host_company_code VARCHAR(10)      NULL,
    host_employee_id  VARCHAR(6)       NULL,
    host_branch_code  VARCHAR(10)      NULL,
    guest_label       VARCHAR(80)      NULL,
    cost_center       VARCHAR(9)       NULL,
    site_code         VARCHAR(10)      NOT NULL,
    served_at        DATETIME2(0)     NOT NULL,
    service_date      DATE             NOT NULL,
    meal_type         TINYINT          NOT NULL,
    identified_by     TINYINT          NOT NULL,
    match_score       DECIMAL(5,4)     NULL,
    operator_user_id  INT              NULL,
    client_uuid       UNIQUEIDENTIFIER NOT NULL,
    synced_at        DATETIME2(0)     NULL,
    CONSTRAINT PK_meal_log PRIMARY KEY CLUSTERED (id),
    CONSTRAINT CK_meal_log_site_code      CHECK (site_code LIKE '[0-9][0-9][0-9][0-9]'),
    CONSTRAINT CK_meal_log_meal_type      CHECK (meal_type IN (1,2,3)),
    CONSTRAINT CK_meal_log_identified_by  CHECK (identified_by IN (1,2,3,4,5)),
    CONSTRAINT CK_meal_log_host CHECK (
        (host_company_code IS NULL AND host_employee_id IS NULL AND host_branch_code IS NULL)
        OR (host_company_code IS NOT NULL AND host_employee_id IS NOT NULL AND host_branch_code IS NOT NULL)),
    CONSTRAINT CK_meal_log_diner CHECK (
        (diner_type = 1 AND company_code IS NOT NULL AND employee_id IS NOT NULL
         AND branch_code IS NOT NULL AND diner_group_id IS NULL)
        OR (diner_type = 2 AND diner_group_id IS NOT NULL AND company_code IS NULL
         AND employee_id IS NULL AND branch_code IS NULL)
        OR (diner_type = 3 AND company_code IS NULL AND employee_id IS NULL

```

```

        AND branch_code IS NULL AND diner_group_id IS NULL)),
CONSTRAINT FK_meal_log_diner_group
    FOREIGN KEY (diner_group_id) REFERENCES GIPP.dbo.meal_diner_group (id)
);
CREATE UNIQUE INDEX UX_meal_log_client ON GIPP.dbo.meal_log (client_uuid);
CREATE INDEX IX_meal_log_dup ON GIPP.dbo.meal_log
    (company_code, employee_id, branch_code, service_date);
CREATE INDEX IX_meal_log_rpt ON GIPP.dbo.meal_log (service_date, site_code)
    INCLUDE (cost_center, company_code);

-- — meal_biometric —————
CREATE TABLE GIPP.dbo.meal_biometric (
    company_code      VARCHAR(10)      NOT NULL,
    employee_id       VARCHAR(6)       NOT NULL,
    branch_code       VARCHAR(10)      NOT NULL,
    embedding         VARBINARY(2048) NOT NULL,
    model_tag         VARCHAR(40)      NOT NULL,
    enrolled_at       DATETIME2(0)     NOT NULL
        CONSTRAINT DF_meal_biometric_enrolled_at DEFAULT (SYSDATETIME()),
    enroll_verified_by TINYINT         NOT NULL,
    consent_at        DATETIME2(0)     NOT NULL,
    consent_version    VARCHAR(20)     NOT NULL,
    consent_ip        VARCHAR(45)      NULL,
    revoked_at        DATETIME2(0)     NULL,
    CONSTRAINT PK_meal_biometric PRIMARY KEY CLUSTERED (company_code, employee_id, branch_code),
    CONSTRAINT CK_meal_biometric_embedding_size CHECK (DATALENGTH(embedding) = 2048),
    CONSTRAINT CK_meal_biometric_verified_by CHECK (enroll_verified_by IN (1,2,3)),
    CONSTRAINT CK_meal_biometric_consent_before_enroll CHECK (consent_at <= enrolled_at),
    CONSTRAINT CK_meal_biometric_revoked_after_enroll
        CHECK (revoked_at IS NULL OR revoked_at >= enrolled_at)
);
CREATE INDEX IX_meal_biometric_revoked ON GIPP.dbo.meal_biometric (revoked_at)
    INCLUDE (company_code, employee_id, branch_code, model_tag);

-- — meal_enroll_token —————
CREATE TABLE GIPP.dbo.meal_enroll_token (
    jti                UNIQUEIDENTIFIER NOT NULL,
    company_code       VARCHAR(10)      NOT NULL,
    employee_id        VARCHAR(6)       NOT NULL,
    branch_code        VARCHAR(10)      NOT NULL,
    issued_at          DATETIME2(0)     NOT NULL
        CONSTRAINT DF_meal_enroll_token_issued_at DEFAULT (SYSDATETIME()),
    expires_at         DATETIME2(0)     NOT NULL,
    issued_by          INT              NOT NULL,
    attempts           TINYINT          NOT NULL
        CONSTRAINT DF_meal_enroll_token_attempts DEFAULT (0),
    consumed_at        DATETIME2(0)     NULL,
    burned_at          DATETIME2(0)     NULL,
    burned_reason       VARCHAR(40)     NULL,
    CONSTRAINT PK_meal_enroll_token PRIMARY KEY CLUSTERED (jti),
    CONSTRAINT CK_meal_enroll_token_expires CHECK (expires_at > issued_at),
    CONSTRAINT CK_meal_enroll_token_attempts CHECK (attempts BETWEEN 0 AND 3),
    CONSTRAINT CK_meal_enroll_token_outcome CHECK (consumed_at IS NULL OR burned_at IS NULL),
    CONSTRAINT CK_meal_enroll_token_burn_reason CHECK (
        (burned_at IS NULL AND burned_reason IS NULL)
        OR (burned_at IS NOT NULL AND burned_reason IS NOT NULL)
);
CREATE UNIQUE INDEX UX_meal_enroll_token_open
    ON GIPP.dbo.meal_enroll_token (company_code, employee_id, branch_code)
    WHERE consumed_at IS NULL AND burned_at IS NULL;

-- — meal_coupon —————
CREATE TABLE GIPP.dbo.meal_coupon (
    id                BIGINT IDENTITY(1,1) NOT NULL,
    nfe_key           CHAR(44)           NOT NULL,
    seq              SMALLINT            NOT NULL,
    meals_authorized SMALLINT            NOT NULL,

```

```

site_code      CHAR(4)      NOT NULL,
coupon_date    DATE         NOT NULL,
service_date   DATE         NOT NULL,
seqproduto     INT          NOT NULL,
tp_emis        CHAR(1)     NOT NULL,
meal_log_id    BIGINT       NULL,
operator_user_id INT        NULL,
redeemed_at    DATETIME2(0) NOT NULL
    CONSTRAINT DF_meal_coupon_redeemed_at DEFAULT (SYSDATETIME()),
CONSTRAINT PK_meal_coupon PRIMARY KEY CLUSTERED (id),
CONSTRAINT UX_meal_coupon_seq UNIQUE (nfe_key, seq),
CONSTRAINT CK_meal_coupon_nfe_key CHECK (nfe_key NOT LIKE '%[^0-9]%'),
CONSTRAINT CK_meal_coupon_authorized CHECK (meals_authorized >= 1),
CONSTRAINT CK_meal_coupon_seq CHECK (seq >= 1 AND seq <= meals_authorized),
CONSTRAINT CK_meal_coupon_tp_emis CHECK (tp_emis IN ('1','9')),
CONSTRAINT FK_meal_coupon_meal_log
    FOREIGN KEY (meal_log_id) REFERENCES GIPP.dbo.meal_log (id)
);
CREATE INDEX IX_meal_coupon_key ON GIPP.dbo.meal_coupon (nfe_key) INCLUDE (seq);
CREATE INDEX IX_meal_coupon_rpt ON GIPP.dbo.meal_coupon (service_date, site_code)
    INCLUDE (nfe_key, tp_emis, meals_authorized);

```

Os nomes de *constraints default* abreviados (**DF_mdg_***) são reconstruções legíveis: na instância eles têm nomes gerados pelo SQL Server. Nenhum *trigger* existe em nenhuma das cinco tabelas.

Anexo B — consultas de verificação

As consultas que sustentam as afirmações deste documento. Todas são somente leitura.

CONFIRMAR QUE OS `CHECK` ESTÃO CONFIÁVEIS

```
-- is_not_trusted = 1 significa que o SQL Server não revalidou as linhas e passou
-- a ignorar a constraint no plano. Se aparecer 1, algo saiu com NOCHECK.
SELECT OBJECT_NAME(parent_object_id) AS tabela, name, is_not_trusted, definition
FROM sys.check_constraints
WHERE OBJECT_NAME(parent_object_id) LIKE 'meal%'
ORDER BY tabela, name;
```

MEDIR A DIVERGÊNCIA DE FUSO DAS SEÇÕES 3.4 E 14.1

```
-- Se o TTL configurado é 7 dias, esta consulta deveria devolver 0 em todas as
-- linhas. Devolve 10845 segundos (3h00m45s) - a diferença UTC vs local.
SELECT jti, issued_at, expires_at,
DATEDIFF(SECOND, DATEADD(DAY, 7, issued_at), expires_at) AS desvio_segundos
FROM GIPP.dbo.meal_enroll_token
ORDER BY issued_at;
```

REFEIÇÕES CUJO `SERVED_AT` (UTC) CAI EM DIA DIFERENTE DE `SERVICE_DATE`

```
SELECT id, served_at, service_date, meal_type, site_code
FROM GIPP.dbo.meal_log
WHERE CAST(served_at AS DATE) <> service_date
ORDER BY served_at;
```

SALDO DE UM CUPOM

```
SELECT nfe_key,
MIN(meals_authorized) AS autorizadas,
COUNT_BIG(*) AS resgatadas,
MIN(meals_authorized) - COUNT_BIG(*) AS saldo,
MAX(seq) AS ultimo_seq
FROM GIPP.dbo.meal_coupon
GROUP BY nfe_key;
```

SITUAÇÃO DOS CADASTROS FACIAIS, SEM TRAZER O VETOR

```
SELECT company_code, employee_id, branch_code, model_tag,
enroll_verified_by, consent_version, enrolled_at, revoked_at,
DATALENGTH(embedding) AS bytes -- deve ser sempre 2048
FROM GIPP.dbo.meal_biometric
ORDER BY enrolled_at DESC;
```

CONVITES ABERTOS POR PESSOA — DEVE DEVOLVER NO MÁXIMO 1 POR TRINCA

```
SELECT company_code, employee_id, branch_code, COUNT_BIG(*) AS abertos
FROM GIPP.dbo.meal_enroll_token
WHERE consumed_at IS NULL AND burned_at IS NULL
GROUP BY company_code, employee_id, branch_code
HAVING COUNT_BIG(*) > 1; -- linhas aqui = índice único filtrado violado
```

COERÊNCIA ENTRE DINER_TYPE E IDENTIFIED_BY (REGRA QUE O BANCO NÃO GARANTE)

```
SELECT id, diner_type, identified_by, match_score
FROM GIPP.dbo.meal_log
WHERE (diner_type = 1 AND identified_by NOT IN (1,2,3))
OR (diner_type = 2 AND identified_by <> 4)
OR (diner_type = 3 AND identified_by <> 5)
OR (match_score IS NOT NULL AND identified_by <> 3);
```

REDESCOBRIR TODA A ESTRUTURA, COLUNA POR COLUNA

```
SELECT t.name AS tabela, c.column_id, c.name AS coluna, ty.name AS tipo,
       c.max_length, c.precision, c.scale, c.is_nullable, c.is_identity,
       OBJECT_DEFINITION(c.default_object_id) AS default_definition
FROM sys.tables t
JOIN sys.columns c ON c.object_id = t.object_id
JOIN sys.types ty ON ty.user_type_id = c.user_type_id
WHERE t.name LIKE 'meal%'
ORDER BY t.name, c.column_id;
```